МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ Федеральное государственное бюджетное образовательное учреждение высшего образования

«СИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ТЕЛЕКОММУНИКАЦИЙ И ИНФОРМАТИКИ»
(СибГУТИ)
Институт вычислительной техники
Кафедра вычислительных систем

ОТЧЕТ

по практической работе № 4
по дисциплине «Основы разработки на Java»

Выполнил: студент гр. ИС-541
Устюжанин Д. К.

Проверил: доцент кафедры ВС
Нигматулин Р. Р.

Новосибирск 2026

Содержание

Содержание **2**
1. Введение **3**
1.1 Цель работы 3
1.2 Предметная область и стек технологий 3
2. Раздел I. Обработка исключений в Java **4**
2.1 Задание 1. Анализ стек-трейса вызовов при исключениях 4
2.2 Задание 2. Классификация и обоснование типов исключений 4
2.3 Задание 3. Кастомные проверяемые исключения и контекстные данные 6
2.4 Задание 4. Анализ поведения конструкций try-finally 7
2.5 Задание 5. Управление ресурсами и подавленные исключения (AutoCloseable) .. 7
2.6 Задание 6. Цепочки исключений (Exception Chaining) 8
2.7 Задание 7. Стратегии обработки ошибок: Optional vs Checked Exception 8
3. Раздел II. Юнит-тестирование и проверка контрактов **9**
3.1 Задание 8. Основы юнит-тестирования и шаблон AAA (Arrange-Act-Assert) 9
3.2 Задание 9. Тестирование граничных условий контрактов 9
3.3 Задание 10. Нормализация данных и параметризованные тесты 9
3.4 Задание 11. Проверка сохранения инвариантов при сбоях 10
4. Раздел III. Проектирование надежных систем и архитектура тестов **11**
4.1 Задание 12. Анализ и устранение причин хрупкости тестов 11
4.2 Задание 13. Тестирование сложных бизнес-правил по таблице решений 11
4.3 Задание 14. Атомарный импорт данных и управление ресурсами 12
4.4 Задание 15. Проектирование системных границ и трансляция ошибок 13
5. Заключение **14**
Приложение А. Полные листинги программы **15**

1. Введение

В рамках практической работы № 2 рассматриваются фундаментальные механизмы обеспечения надежности и корректности программных систем на языке Java: система обработки исключительных ситуаций (Exception Handling), автоматическое управление ресурсами (try-with-resources, AutoCloseable), а также современная методология юнит-тестирования с использованием фреймворка JUnit 5.

Практическая реализация и тестирование выполняются на основе предметной модели учебного центра **CourseHub**, развивающей концепции, заложенные в практической работе № 1.

1.1 Цель работы

Освоить практические навыки работы с механизмом исключений в Java, научиться проектировать пользовательские иерархии исключений, правильно применять конструкции try-catch-finally и try-with-resources, освоить технику Exception Chaining. Изучить методологию юнит-тестирования (AAA), написание граничных и параметризованных тестов, устранение хрупкости тестов (test flakiness), а также проектирование робастных сервисов с атомарными операциями и обработкой ошибок на границах системы.

1.2 Предметная область и стек технологий

Предметная модель включает ключевые сущности системы CourseHub:

- **`src.Course`** — учебный курс (идентификатор, наименование, длительность, пройденные часы, вместимость, статус, список зачисленных студентов);
- **`CourseCode`** — доменный объект кода курса с автоматической нормализацией;
- **`EnrollmentService / CourseStore / CourseImporter`** — сервисный слой регистрации, хранения и импорта данных;
- **`CourseHub`** — главный фасад системы, транслирующий доменные ошибки в фиксированные коды ErrorCode.

Стек технологий: Java 17+, JUnit 5 (Jupiter), Maven.

2. Раздел I. Обработка исключений в Java

2.1 Задание 1. Анализ стек-трейса вызовов при исключениях

Продемонстрирована работа механизма формирования стек-трейса при глубокой цепочке вызовов методов (main -> a -> b -> c). Стек-трейс фиксирует последовательность кадров (stack frames) на момент возникновения исключения, позволяя точно определить источник и контекст возникновения ошибки.

Листинг 1 — Демонстрация формирования стек-трейса вызовов

```
public class StackTraceDemo {
    public static void main(String[] args) {
        a();
    }
    static void a() { b(); }
    static void b() { c(); }
    static void c() {
        int x = 10;
    }
}
```

2.2 Задание 2. Классификация и обоснование типов исключений

В Java иерархия исключений делится на три ключевых категории: checked (проверяемые), unchecked (RuntimeException) и Error.

Таблица 1 — Классификация исключительных ситуаций

№	Ситуация	Тип исключения	Обоснование выбора
1	Неверный аргумент метода	`unchecked (IllegalArgumentException)`	Ошибка вызывающего кода (нарушение предусловия).
2	Отсутствующий файл	`checked (FileNotFoundException / IOException)`	Внешний ресурс может отсутствовать, вызывающий обязан обработать сценарий.

№	Ситуация	Тип исключения	Обоснование выбора
3	Нарушение состояния объекта	`unchecked (IllegalStateException)`	Объект находится в невалидном состоянии для выполнения операции.
4	Сетевой отказ	`checked (IOException)`	Предсказуемый инфраструктурный сбой внешней среды.
5	Ошибка конфигурации	`checked (ConfigurationException)`	Ожидаемая проблема при старте приложения, требующая коррекции.
6	Недостаток памяти	`Error (OutOfMemoryError)`	Фатальный сбой виртуальной машины, восстановление невозможно.
7	Некорректная запись при импорте	`checked (CourseImportException)`	Ожидаемая ошибка входных данных с необходимостью указывать номер строки.
8	Нет прав доступа к файлу	`checked (AccessDeniedException)`	Инфраструктурный сбой доступа к внешней системе.
9	Таймаут сети	`checked (TimeoutException)`	Ожидаемый отказ сетевого взаимодействия.

№	Ситуация	Тип исключения	Обоснование выбора
10	Программная ошибка: null	`unchecked (NullPointerException)`	Дефект реализации (баг в коде).

Эмпирическое правило:

- **Checked** — ожидаемые проблемы внешней среды или данных, из которых система может и должна восстановиться.
- **Unchecked** — ошибки программиста (баги), нарушение инвариантов и контрактов.
- **Error** — критические сбои среды исполнения (JVM).

2.3 Задание 3. Кастомные проверяемые исключения и контекстные данные

Создано доменное проверяемое исключение `EnrollmentRejectedException`, содержащее перечисление причин отказа `ReasonCode` (`COURSE_FULL`, `PREREQUISITE_MISSING`, `ALREADY_ENROLLED`, `COURSE_CLOSED`), а также контекстные идентификаторы `studentID` и `courseID`. Переопределен метод `toString()` для предотвращения утечки чувствительных данных в логи.

Листинг 2 — Доменное проверяемое исключение EnrollmentRejectedException

```
public class EnrollmentRejectedException extends Exception {
    public enum ReasonCode {
        COURSE_FULL, PREREQUISITE_MISSING, ALREADY_ENROLLED,
COURSE_CLOSED
    }
    private final long studentID;
    private final long courseID;
    private final ReasonCode reasonCode;
    public EnrollmentRejectedException(long studentID, long courseID,
ReasonCode reasonCode) {
        super("Enrollment rejected: " + reasonCode);
        this.studentID = studentID;
        this.courseID = courseID;
        this.reasonCode = reasonCode;
    }
    @Override
    public String toString() {
        return "EnrollmentRejectedException{reasonCode=" + reasonCode +
"}";
```

```
    }
}
```

2.4 Задание 4. Анализ поведения конструкций try-finally

Исследованы особенности работы блока `finally`:

- Блок `finally` выполняется всегда перед выходом из метода, даже при наличии `return` в блоке `try`.
- Если `finally` содержит явный оператор `return`, он перекрывает значение возврата или исключение из блока `try`, что приводит к «потере» ошибок (exception swallowing).

2.5 Задание 5. Управление ресурсами и подавленные исключения (AutoCloseable)

Использование конструкции `try-with-resources` гарантирует вызов метода `close()` ресурсов, реализующих `AutoCloseable`. При совпадении сбоев во время выполнения основной операции и при закрытии ресурса, основное исключение становится первичным (primary), а ошибка закрытия сохраняется в списке подавленных исключений (`e.getSuppressed()`).

Листинг 3 — Реализация AutoCloseable и извлечение подавленных исключений

```
class LoggingResource implements AutoCloseable {
    private final boolean failOnUse;
    private final boolean failOnClose;
    LoggingResource(boolean failOnUse, boolean failOnClose) {
        this.failOnUse = failOnUse;
        this.failOnClose = failOnClose;
    }
    void use() {
        if (failOnUse) throw new RuntimeException("use failed");
    }
    @Override
    public void close() {
        if (failOnClose) throw new RuntimeException("close failed");
    }
}
public class Main {
    public static void main(String[] args) {
        try (LoggingResource r = new LoggingResource(true, true)) {
            r.use();
        } catch (RuntimeException e) {
            System.out.println("primary = " + e.getMessage());
            for (Throwable suppressed : e.getSuppressed()) {
```

```
                System.out.println("suppressed = " +
suppressed.getMessage());
            }
        }
    }
}
```

2.6 Задание 6. Цепочки исключений (Exception Chaining)

Реализовано обертывание низкоуровневых ошибок (например, `NumberFormatException`) в доменные проверяемые исключения `CourseImportException`. Сохранение исходного исключения в качестве причины (`cause`) обеспечивает полную трассируемость ошибок без нарушения абстракции слоев.

Листинг 4 — Трансляция исключений с сохранением причины (Exception Chaining)

```
public class ParseRecord {
    public static src.Course parseRecord(String line, int recordNumber)
throws CourseImportException {
        try {
            int duration = Integer.parseInt(line.trim());
            return new src.Course(recordNumber, "Imported", duration);
        } catch (NumberFormatException e) {
            throw new CourseImportException(recordNumber, "Invalid
duration in record " + recordNumber, e);
        }
    }
}
```

2.7 Задание 7. Стратегии обработки ошибок: Optional vs Checked Exception

Сравнены два подхода к обработке отсутствия или отказа:

- **`Optional<T>`** используется для штатных бизнес-сценариев, когда отсутствие результата является нормальным ожидаемым исходом (например, поиск курса по ID);
- **Checked Exception** используется при невозможности продолжения операции из-за нарушения бизнес-правил или сбоев (например, отказ в зачислении).

3. Раздел II. Юнит-тестирование и проверка контрактов

3.1 Задание 8. Основы юнит-тестирования и шаблон AAA (Arrange-Act-Assert)

Все тесты структурированы по стандартному шаблону AAA:

- **Arrange**: подготовка тестового окружения и входных данных;
- **Act**: выполнение тестируемой операции;
- **Assert**: проверка результатов и инвариантов.

3.2 Задание 9. Тестирование граничных условий контрактов

Протестированы граничные значения длительности курса (`MIN_DURATION_HOURS = 1`, `MAX_DURATION_HOURS = 100`). Написание тестов только на «типичные» значения не гарантирует правильность границ. Проверены случаи: `MIN - 1` (отклоняется), `MIN` (принимается), `MAX` (принимается), `MAX + 1` (отклоняется).

3.3 Задание 10. Нормализация данных и параметризованные тесты

Разработан класс `CourseCode` с нормализацией строк (удаление пробелов, дефисов, приведение к верхнему регистру). Использованы параметризованные тесты `@ParameterizedTest` с источниками `@CsvSource` и `@MethodSource`, что позволило проверить множественные наборы невалидных и валидных кодов без дублирования тестовой логики.

Листинг 5 — Нормализация CourseCode и параметризованное тестирование

```
public class CourseCode {
    private final String value;
    private CourseCode(String value) { this.value = value; }
    public static CourseCode parse(String raw) {
        String normalized = normalize(raw);
        if (normalized.isEmpty()) throw new
IllegalArgumentException("code must not be blank");
        return new CourseCode(normalized);
    }
    public static String normalize(String raw) {
        return raw.trim().toUpperCase(Locale.ROOT).replace(" ",
"").replace("-", "").replace("_", "");
    }
}
class CourseCodeTest {
    @ParameterizedTest(name = "normalize(\"{0}\") -> \"{1}\"")
```

```
    @CsvSource({"JAVA101, JAVA101", "' java-101 ', JAVA101", "'java_101',
JAVA101"})
    void normalizesRawCodeToCanonicalForm(String raw, String expected) {
        assertEquals(expected, CourseCode.parse(raw).value());
    }
}
```

3.4 Задание 11. Проверка сохранения инвариантов при сбоях

Доказана важность проверки состояния объектов после сбоя. При отклонении операции (например, попытке зачисления на полный курс или превышении часов) объект должен оставаться в исходном валидном состоянии без частичных изменений.

Листинг 6 — Проверка сохранения инвариантов объектов после сбоя

```
@Test
void enroll_rejectsDuplicate_andKeepsRoster() throws
EnrollmentRejectedException {
    Course course = new Course(2, "Kotlin", 20);
    Enrollment enrollment = new Enrollment(course, 2);
    enrollment.enroll(100);
    EnrollmentRejectedException e = assertThrows(
        EnrollmentRejectedException.class,
        () -> enrollment.enroll(100));
    assertEquals(EnrollmentRejectedException.ReasonCode.ALREADY_ENROLLED,
e.reasonCode());
    assertEquals(List.of(100L), enrollment.enrolledStudents());
    assertEquals(1, enrollment.seatsLeft());
}
```

4. Раздел III. Проектирование надежных систем и архитектура тестов

4.1 Задание 12. Анализ и устранение причин хрупкости тестов

В классе `BrokenCourseTest` выявлены 4 основные причины хрупкости тестов (test flakiness):

1. Зависимость от порядка выполнения тестов;
2. Утечка глобального/статического состояния (`static` поля);
3. Зависимость от текущей рабочей директории при чтении файлов;
4. Жесткая привязка к exact-тексту метода `toString()`.

В `RobustCourseTest` данные проблемы устранены с использованием изолированных экземпляров репозиториев, временных директорий `@TempDir` и проверки доменных свойств вместо строк.

Листинг 7 — Устранение причин хрупкости тестов (RobustCourseTest)

```
class RobustCourseTest {
    @Test
    void savedRecordIsNotVisibleInAnotherRepository() {
        CourseRepository repository = new CourseRepository();
        repository.save(new CourseRecord(1, "Java", 32));
        CourseRepository another = new CourseRepository();
        assertNull(another.find(1), "state leaked between tests");
    }
    @Test
    void catalog_loadsRecordsFromGivenPath(@TempDir Path tmp) throws
IOException {
        Path file = tmp.resolve("courses.csv");
        Files.writeString(file, "1;Java;32\n2;Kotlin;24\n");
        List<CourseRecord> records = CourseCatalog.loadCsv(file);
        assertEquals(2, records.size());
    }
}
```

4.2 Задание 13. Тестирование сложных бизнес-правил по таблице решений

Для процесса зачисления (`EnrollmentService`) составлена и протестирована таблица решений, учитывающая 4 фактора: статус курса, наличие свободных мест, наличие пререквизитов и повторную запись. Порядок проверок зафиксирован в коде и покрыт тестами с приоритетом причин отказа.

Листинг 8 — Тестирование таблицы решений для процесса зачисления

```
@ParameterizedTest
@MethodSource("cases")
void enrollmentDecision(Course.Status status, boolean hasSeats, boolean
hasPrereq, boolean repeat, Outcome expected) {
    Course course = new Course(1, "Java", 32);
    course.setStatus(status);
    course.setCapacity(hasSeats ? 2 : 0);
    course.setHasPrerequisite(hasPrereq);
    if (repeat) course.addEnrolled(42);
    EnrollmentService service = new EnrollmentService();
    if (expected == Outcome.ENROLL_OK) {
        assertDoesNotThrow(() -> service.enroll(course, 43));
    } else {
        EnrollmentRejectedException e =
assertThrows(EnrollmentRejectedException.class, () ->
service.enroll(course, 42));
        assertEquals(expected.reason(), e.reasonCode());
    }
}
```

4.3 Задание 14. Атомарный импорт данных и управление ресурсами

Реализован `CourseImporter` с атомарной публикацией результатов. Данные накапливаются во временном буфере и публикуются в `CourseStore` только при успешном чтении всех строк. В случае ошибки на любой строке публикация не производится, а ресурсы гарантированно освобождаются.

Листинг 9 — Реализация CourseImporter с атомарной публикацией данных

```
public class CourseImporter {
    private final CourseStore store;
    public CourseImporter(CourseStore store) { this.store = store; }
    public int importFrom(Reader reader) throws IOException,
CourseImportException {
        List<Course> buffer = new ArrayList<>();
        BufferedReader br = new BufferedReader(reader);
        String line;
        int recordNumber = 0;
        try (br) {
            while ((line = br.readLine()) != null) {
                recordNumber++;
                buffer.add(parseRecord(line, recordNumber));
            }
            store.publish(buffer);
            return buffer.size();
        }
```

```
    }
}
```

4.4 Задание 15. Проектирование системных границ и трансляция ошибок

Фасад `CourseHub` формирует изолированную границу системы. Все внутренние доменные и инфраструктурные исключения перехватываются и транслируются в объекты `ImportResult` / `EnrollResult` с фиксированными кодами ошибок `ErrorCode` (`ENROLL_CLOSED`, `ENROLL_FULL`, `IMPORT_MALFORMED_RECORD` и др.), предотвращая просачивание внутренних исключений на внешний слой.

Листинг 10 — Реализация CourseHub и результат трансляции ошибок EnrollResult

```
public class CourseHub {
    private final CourseStore store;
    private final EnrollmentService enrollment;
    public EnrollResult enroll(long studentId, long courseId) {
        try {
            Course course = store.get(courseId);
            if (course == null) return
EnrollResult.fail(ErrorCode.HUB_INVALID_ARGUMENT, "not found", null);
            enrollment.enroll(course, studentId);
            return EnrollResult.ok();
        } catch (EnrollmentRejectedException e) {
            ErrorCode code = switch (e.reasonCode()) {
                case COURSE_CLOSED -> ErrorCode.ENROLL_CLOSED;
                case COURSE_FULL -> ErrorCode.ENROLL_FULL;
                case PREREQUISITE_MISSING ->
ErrorCode.ENROLL_PREREQ_MISSING;
                case ALREADY_ENROLLED -> ErrorCode.ENROLL_ALREADY;
            };
            return EnrollResult.fail(code, e.getMessage(), e.getCause());
        }
    }
}
```

5. Заключение

В ходе выполнения практической работы № 2 были достигнуты следующие результаты:

1. Изучены механизмы работы исключений в Java, принципы разделения на проверяемые, непроверяемые исключения и ошибки, освоен паттерн Exception Chaining.
2. Освоено автоматическое управление ресурсами через `AutoCloseable` и `try-with-resources` с учетом подавленных исключений.
3. Изучена методология юнит-тестирования с помощью JUnit 5 (AAA, параметризованные тесты, граничный анализ).
4. Освоены методы устранения хрупкости тестов и обеспечения изоляции тестового состояния.
5. Спроектирован сервисный слой системы `CourseHub` с атомарным импортом данных и строгой трансляцией ошибок на системных границах.

Приложение А. Полные листинги программы

Листинг 11 — StackTraceDemo.java (задание 1)

```
public class StackTraceDemo {
    public static void main(String[] args) {
        a();
    }
    static void a() {
        b();
    }
    static void b() {
        c();
    }
    static void c() {
        // int x = 10/0;
        int x = 10;
    }
}
```

Листинг 12 — EnrollmentRejectedException.java (задания 3, 11, 13, 15)

```
public class EnrollmentRejectedException extends Exception {
    public enum ReasonCode {
        COURSE_FULL,
        PREREQUISITE_MISSING,
        ALREADY_ENROLLED,
        COURSE_CLOSED
    }
    private final long studentID;
    private final long courseID;
    private final ReasonCode reasonCode;
    public EnrollmentRejectedException(long studentID, long courseID,
ReasonCode reasonCode) {
        super("Enrollment rejected: " + reasonCode);
        this.studentID = studentID;
        this.courseID = courseID;
        this.reasonCode = reasonCode;
    }
    public EnrollmentRejectedException(long studentID, long courseID,
ReasonCode reasonCode, Throwable cause) {
        super("Enrollment rejected: " + reasonCode, cause);
        this.studentID = studentID;
        this.courseID = courseID;
        this.reasonCode = reasonCode;
    }
    public long studentID() { return studentID; }
    public long courseID() { return courseID; }
```

```
    public ReasonCode reasonCode() { return reasonCode; }
    @Override
    public String toString() {
        return "EnrollmentRejectedException{reasonCode=" + reasonCode +
"}";
    }
}
```

Листинг 13 — TryFinallyExamples.java (задание 4)

```
static int example1() {
    try {
        return 1;
    } finally {
        System.out.println("finally");
    }
}
static int example2() {
    try {
        return 1;
    } finally {
        return 2;
    }
}
static int example3() {
    try {
        throw new RuntimeException("A");
    } finally {
        return 42;
    }
}
```

Листинг 14 — LoggingResource.java (задание 5)

```
class LoggingResource implements AutoCloseable {
    private final boolean failOnUse;
    private final boolean failOnClose;
    LoggingResource(boolean failOnUse, boolean failOnClose) {
        this.failOnUse = failOnUse;
        this.failOnClose = failOnClose;
        System.out.println("open");
    }
    void use() {
        System.out.println("use");
        if (failOnUse) {
            throw new RuntimeException("use failed");
        }
    }
```

```
    @Override
    public void close() {
        System.out.println("close");
        if (failOnClose) {
            throw new RuntimeException("close failed");
        }
    }
}
```

Листинг 15 — CourseImportException.java (задания 6, 14, 15)

```
public class CourseImportException extends Exception {
    private final int recordNumber;
    public CourseImportException(int recordNumber, String message,
Throwable cause) {
        super(message, cause);
        this.recordNumber = recordNumber;
    }
    public int recordNumber() {
        return recordNumber;
    }
}
```

Листинг 16 — ParseRecord.java (задание 6)

```
import src.Course;
public class ParseRecord {
    public static Course parseRecord(String line, int recordNumber)
throws CourseImportException {
        try {
            int duration = ParseDuration.parseDuration(line);
            return new Course(recordNumber, "Imported", duration);
        } catch (NumberFormatException e) {
            throw new CourseImportException(
                recordNumber,
                "Invalid duration in record " + recordNumber,
                e
            );
        }
    }
}
```

Листинг 17 — FindCourse.java (задание 7)

```
import java.util.Optional;
import src.Course;
public class FindCourse {
    public static Optional<Course> find(long id) {
```

```
        if (id == 1) {
            return Optional.of(new Course(1, "Java", 10));
        }
        return Optional.empty();
    }
}
```

Листинг 18 — CourseTest.java (задание 8)

```
import org.junit.jupiter.api.Test;
import src.Course;
import static org.junit.jupiter.api.Assertions.*;
class CourseTest {
    @Test
    void completeHours_increasesCompletedHours_whenEnoughRemaining() {
        // Arrange
        Course course = new Course(1, "Java", 10, 2);
        // Act
        course.completeHours(3);
        // Assert
        assertEquals(5, course.completedHours(), "completed hours should
increase by 3");
    }
}
```

Листинг 19 — DurationBoundaryTest.java (задание 9)

```
import org.junit.jupiter.api.Test;
import src.Course;
import static org.junit.jupiter.api.Assertions.*;
class DurationBoundaryTest {
    @Test
    void constructor_rejectsDurationBelowMinimum() {
        int tooSmall = Course.MIN_DURATION_HOURS - 1;
        IllegalArgumentException e =
assertThrows(IllegalArgumentException.class, () -> new Course(1, "Java",
tooSmall));
        assertEquals("durationHours must be in [1, 100]",
e.getMessage());
    }
    @Test
    void constructor_acceptsMinimumDuration() {
        Course course = new Course(1, "Java", Course.MIN_DURATION_HOURS);
        assertEquals(1, course.durationHours());
    }
    @Test
    void constructor_acceptsMaximumDuration() {
        Course course = new Course(1, "Java", Course.MAX_DURATION_HOURS);
```

```
        assertEquals(100, course.durationHours());
    }
    @Test
    void constructor_rejectsDurationAboveMaximum() {
        int tooBig = Course.MAX_DURATION_HOURS + 1;
        IllegalArgumentException e =
assertThrows(IllegalArgumentException.class, () -> new Course(1, "Java",
tooBig));
        assertEquals("durationHours must be in [1, 100]",
e.getMessage());
    }
}
```

Листинг 20 — CourseCode.java (задания 10, 15)

```
import java.util.Locale;
public class CourseCode {
    public static final int MIN_LENGTH = 3;
    public static final int MAX_LENGTH = 10;
    private final String value;
    private CourseCode(String value) { this.value = value; }
    public static CourseCode parse(String raw) {
        String normalized = normalize(raw);
        if (normalized.isEmpty()) throw new
IllegalArgumentException("code must not be blank");
        if (normalized.length() < MIN_LENGTH || normalized.length() >
MAX_LENGTH) {
            throw new IllegalArgumentException("code length must be in ["
+ MIN_LENGTH + ", " + MAX_LENGTH + "]");
        }
        for (int i = 0; i < normalized.length(); i++) {
            char c = normalized.charAt(i);
            boolean latinLetterOrDigit = (c >= 'A' && c <= 'Z') || (c >=
'0' && c <= '9');
            if (!latinLetterOrDigit) {
                throw new IllegalArgumentException("code must contain
only latin letters and digits, got '" + c + "'");
            }
        }
        return new CourseCode(normalized);
    }
    public static String normalize(String raw) {
        return raw.trim().toUpperCase(Locale.ROOT).replace(" ",
"").replace("-", "").replace("_", "");
    }
    public String value() { return value; }
    @Override
    public boolean equals(Object o) {
```

```
        if (this == o) return true;
        if (!(o instanceof CourseCode other)) return false;
        return value.equals(other.value);
    }
    @Override
    public int hashCode() { return value.hashCode(); }
    @Override
    public String toString() { return value; }
}
```

Листинг 21 — CourseStateAfterFailureTest.java (задание 11)

```
import org.junit.jupiter.api.Test;
import src.Course;
import static org.junit.jupiter.api.Assertions.*;
class CourseStateAfterFailureTest {
    @Test
    void completeHours_rejectsMoreThanRemaining_andKeepsCourseState() {
        Course course = new Course(1, "Java", 10, 2);
        IllegalStateException e =
assertThrows(IllegalStateException.class, () ->
course.completeHours(999));
        assertEquals("cannot complete 999 h, only 8 h left",
e.getMessage());
        assertEquals(2, course.completedHours());
    }
}
```

Листинг 22 — RobustCourseTest.java (задание 12)

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;
class RobustCourseTest {
    @Test
    void find_returnsRecordSavedInSameRepository() {
        CourseRepository repository = new CourseRepository();
        repository.save(new CourseRecord(1, "Java", 32));
        CourseRecord found = repository.find(1);
        assertNotNull(found);
        assertEquals("Java", found.title());
    }
    @Test
```

```
    void catalog_loadsRecordsFromGivenPath(@TempDir Path tmp) throws
IOException {
        Path file = tmp.resolve("courses.csv");
        Files.writeString(file, "1;Java;32\n2;Kotlin;24\n");
        List<CourseRecord> records = CourseCatalog.loadCsv(file);
        assertEquals(2, records.size());
    }
}
```

Листинг 23 — EnrollmentDecisionTest.java (задание 13)

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.Arguments;
import org.junit.jupiter.params.provider.MethodSource;
import src.Course;
import java.util.stream.Stream;
import static org.junit.jupiter.api.Assertions.*;
class EnrollmentDecisionTest {
    @ParameterizedTest
    @MethodSource("cases")
    void enrollmentDecision(Course.Status status, boolean hasSeats,
boolean hasPrereq, boolean repeat, Outcome expected) {
        Course course = new Course(1, "Java", 32);
        course.setStatus(status);
        course.setCapacity(hasSeats ? 2 : 0);
        course.setHasPrerequisite(hasPrereq);
        if (repeat) course.addEnrolled(42);
        EnrollmentService service = new EnrollmentService();
        if (expected == Outcome.ENROLL_OK) {
            assertDoesNotThrow(() -> service.enroll(course, 43));
        } else {
            EnrollmentRejectedException e =
assertThrows(EnrollmentRejectedException.class, () ->
service.enroll(course, 42));
            assertEquals(expected.reason(), e.reasonCode());
        }
    }
    static Stream<Arguments> cases() {
        return Stream.of(
            Arguments.of(Course.Status.OPEN, true, true, false,
Outcome.ENROLL_OK),
            Arguments.of(Course.Status.CLOSED, true, true, false,
Outcome.REJECT_CLOSED),
            Arguments.of(Course.Status.OPEN, false, true, false,
Outcome.REJECT_FULL),
            Arguments.of(Course.Status.OPEN, true, false, false,
Outcome.REJECT_PREREQ),
```

```
            Arguments.of(Course.Status.OPEN, true, true, true,
Outcome.REJECT_ALREADY)
        );
    }
}
```

Листинг 24 — CourseImporter.java (задания 14, 15)

```
import src.Course;
import java.io.BufferedReader;
import java.io.IOException;
import java.io.Reader;
import java.util.ArrayList;
import java.util.List;
public class CourseImporter {
    private final CourseStore store;
    public CourseImporter(CourseStore store) { this.store = store; }
    public int importFrom(Reader reader) throws IOException,
CourseImportException {
        List<Course> buffer = new ArrayList<>();
        BufferedReader br = new BufferedReader(reader);
        String line;
        int recordNumber = 0;
        try (br) {
            while ((line = br.readLine()) != null) {
                recordNumber++;
                String trimmed = line.trim();
                if (trimmed.isEmpty() || trimmed.startsWith("#"))
continue;
                try {
                    buffer.add(parseRecord(trimmed, recordNumber));
                } catch (Exception ex) {
                    throw new CourseImportException(recordNumber,
"Invalid record at line " + recordNumber, ex);
                }
            }
            store.publish(buffer);
            return buffer.size();
        }
    }
}
```

Листинг 25 — CourseHub.java (задание 15)

```
public class CourseHub {
    private final CourseImporter importer;
    private final CourseStore store;
    private final EnrollmentService enrollment;
```

```
    public CourseHub(CourseImporter importer, CourseStore store,
EnrollmentService enrollment) {
        this.importer = importer;
        this.store = store;
        this.enrollment = enrollment;
    }
    public EnrollResult enroll(long studentId, long courseId) {
        try {
            src.Course course = store.get(courseId);
            if (course == null) return
EnrollResult.fail(ErrorCode.HUB_INVALID_ARGUMENT, "course not found",
null);
            enrollment.enroll(course, studentId);
            return EnrollResult.ok();
        } catch (EnrollmentRejectedException e) {
            ErrorCode code = switch (e.reasonCode()) {
                case COURSE_CLOSED -> ErrorCode.ENROLL_CLOSED;
                case COURSE_FULL -> ErrorCode.ENROLL_FULL;
                case PREREQUISITE_MISSING ->
ErrorCode.ENROLL_PREREQ_MISSING;
                case ALREADY_ENROLLED -> ErrorCode.ENROLL_ALREADY;
            };
            return EnrollResult.fail(code, e.getMessage(), e.getCause());
        }
    }
}
```